

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 8
з курсу “Життєвий цикл програмного забезпечення. Управління ІТ
проектами”
на тему: “Неперервна інтеграція та неперервна доставка (CI/CD).
Впровадження CI/CD використовуючи веб-сервіс GitHub actions
workflow”

Виконав:

Студент. гр. ФеП-41 Фіняк Олег

Перевірив:

Франів В.А.

Мета роботи: Мета лабораторної роботи полягає в тому, щоб студенти ознайомилися з процесом впровадження безперервної інтеграції та безперервної доставки (CI/CD) у реальному проекті за допомогою популярного веб ресурсу GitHub actions workflow.

Вимоги до результатів виконання лабораторної роботи:

4. Продемонструвати зконфігурований GitHub actions workflow для обраного проекту.
5. Описати створений GitHub actions workflow. Обґрунтувати кожен крок у Actions workflow
6. Проаналізувати плюси та мінуси використання програмного пакету Jenkins для впровадження CI/CD процесів.

Порядок виконання роботи:

1. Освоїти вище наведений теоретичний матеріал.
2. Ознайомитися з веб ресурсом GitHub (якщо Ви ще не зареєструвалися, зареєструйтесь використовуючи університетську емейл адресу).
3. Для існуючого репозиторію з проектом створіть GitHub action з наступними етапами: збирання проекту(вихідний код знаходиться у GitHub репозиторії), запуск тестів.

...

Lab_6

Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

More

Watch 0

0 stars

0 forks

0 watching

2 Branches

0 Tags

Activity

Public repository

main

Go to file

+

<> Code

Finiik

Merge pull request #1 from Finiik/brach-1

d4d4e46 · 2 hours ago

s-tools

smth

2 hours ago

LOlloI.docx

Test commit for branch-1

2 hours ago

3Bit_1.docx

smth

2 hours ago

README

...

Lab_6

Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

More

Get started with GitHub Actions

Build, test, and deploy your code. Make code reviews, branch management, and issue triaging work the way you want. Select a workflow to get started.

Skip this and [set up a workflow yourself](#) →

Search workflows

... / Lab_6



[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [More](#)

← Files

Lab_6 / .github / workflows

main.yml in main

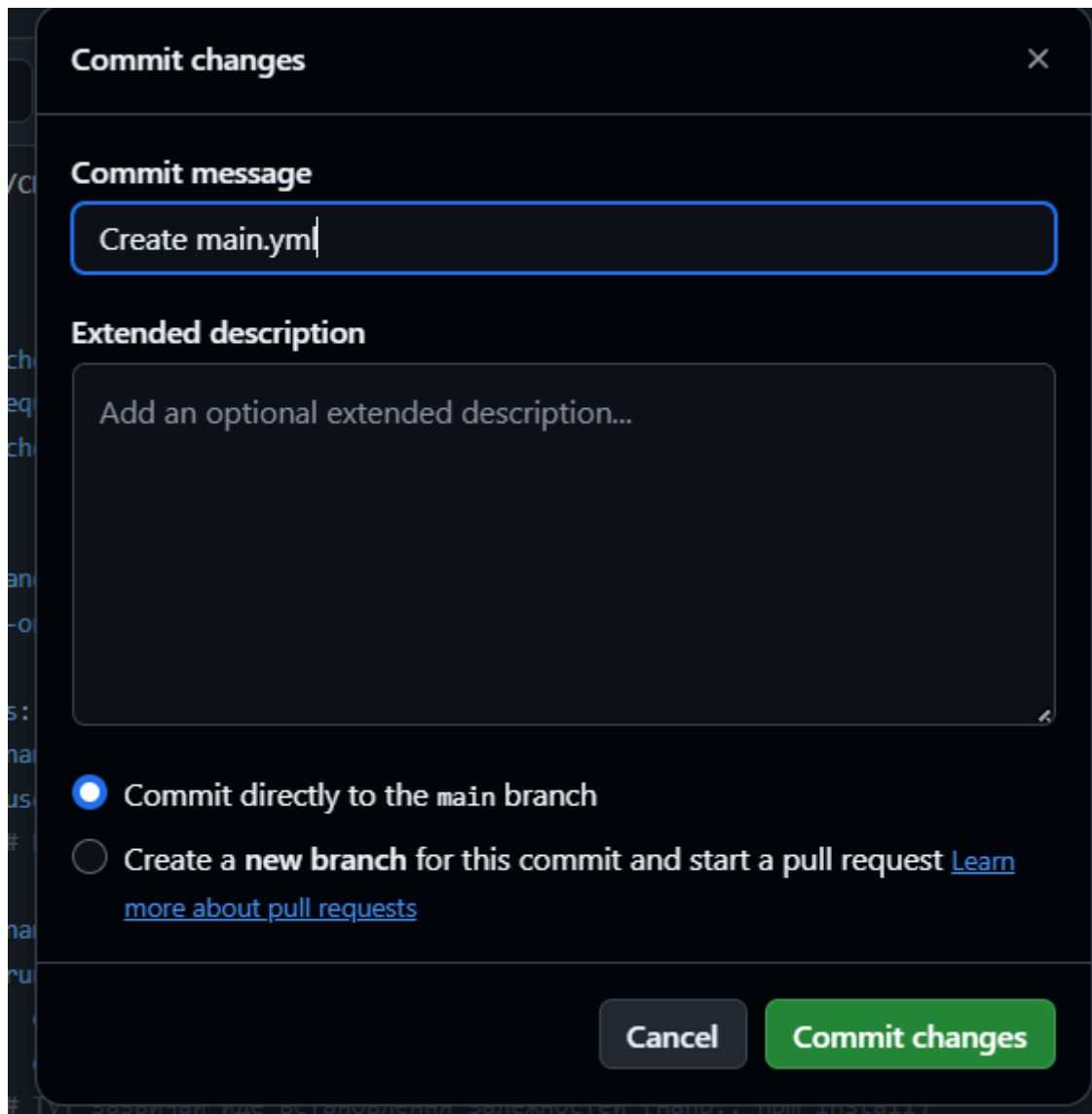
Cancel changes

Commit changes...

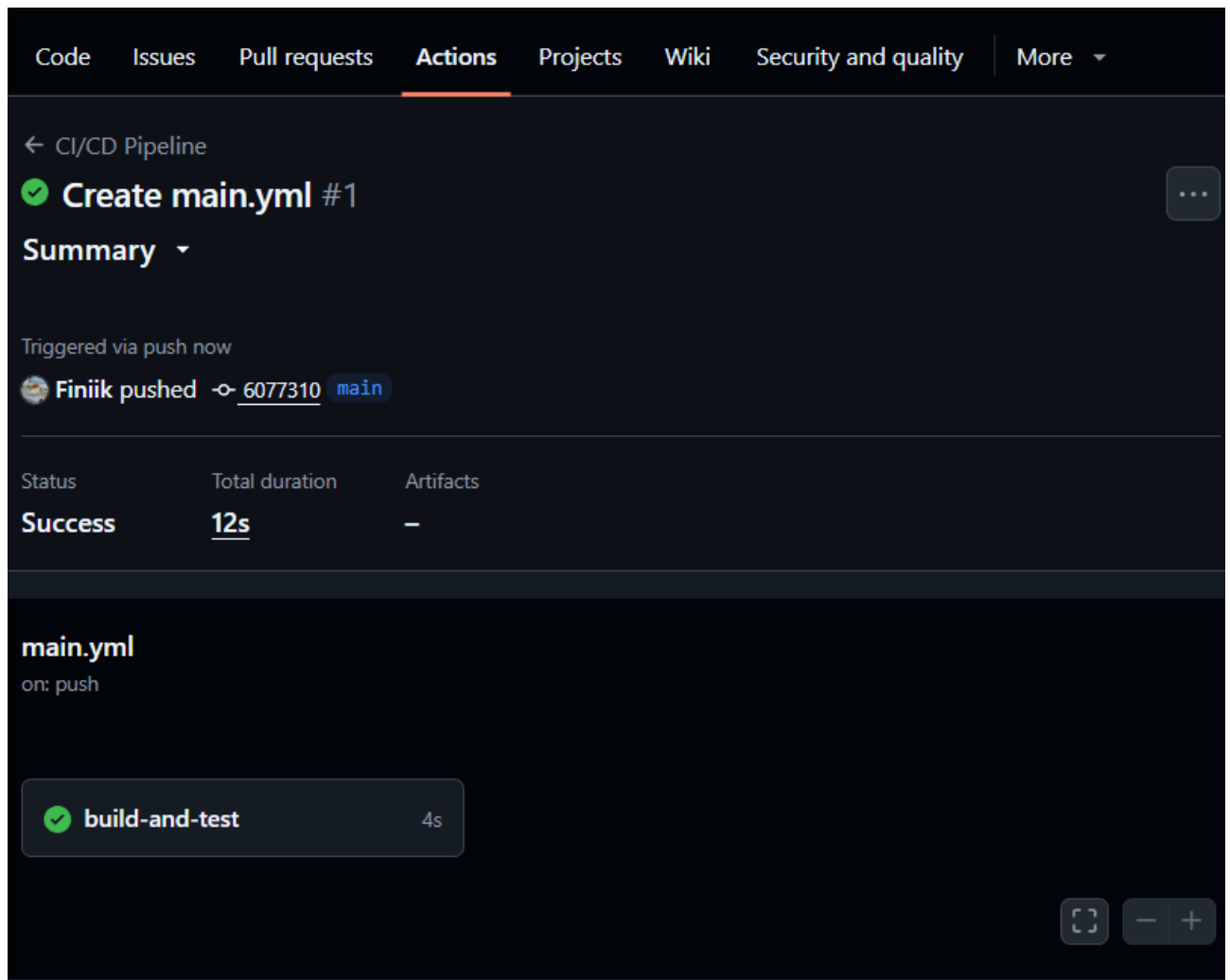
Edit Preview

Spaces 2 No wrap

```
1  name: CI/CD Pipeline
2
3  on:
4    push:
5      branches: [ "main", "master" ]
6    pull_request:
7      branches: [ "main", "master" ]
8
9  jobs:
10   build-and-test:
11     runs-on: ubuntu-latest # Використовуємо хмарний сервер на Linux
12
13     steps:
14       - name: Checkout repository
15         uses: actions/checkout@v3
16         # Цей крок завантажує твій код на віртуальний сервер
17
18       - name: Build Project
19         run: |
20           echo "Збирання проекту..."
21           echo "Проект успішно зібрано!"
22           # Тут зазвичай йде встановлення залежностей (напр., npm install)
23
24       - name: Run Tests
```



4. Успішно виконайте створений GitHub action.



Висновок (Порівняння Jenkins та GitHub Actions): Спираючись на досвід виконання попередньої та поточної лабораторних робіт, можна зробити наступне порівняння:

Інфраструктура: Jenkins вимагає виділеного сервера, встановлення Java та ручного налаштування (що ми бачили у ЛР №7). GitHub Actions є "хмарним" (SaaS) рішенням, яке працює "з коробки" прямо в репозиторії, не вимагаючи підтримки серверів.

Конфігурація: Jenkins використовує Groovy для написання Jenkinsfile, що може бути складним для початківців. GitHub Actions використовує більш читабельний формат YAML.

Інтеграція: GitHub Actions безшовно інтегрований у GitHub, тоді як Jenkins потрібно зв'язувати з репозиторієм через вебхуки та токени доступу.

Підсумок: Для невеликих, середніх та Open Source проектів GitHub Actions є значно зручнішим та швидшим інструментом для впровадження CI/CD процесів. Jenkins залишається актуальним для великих корпоративних проектів зі складною, специфічною інфраструктурою та високими вимогами до безпеки (розгортання у закритому контурі).

Література:

- 16.Scott Chacon, Ben Straub. Pro Git. Apress. 2014. 456 p.
- 17.Jon Loeliger, Matthew McCullough. Version Control with Git. O'Reilly Media. 2012. 456 p.
- 18.Ryan Hodson. Git Essentials. Packt Publishing. 2015. 194 p.
- 19.Roger Dudler. Git Pocket Guide. O'Reilly Media. 2013. 234 p.
- 20.Seth Robertson, Steve Oualline. Practical Git: For Everyday Use. No Starch Press. 2014. 300 p